补充材料：完整库存、费用与同源续行的证明

Complete Proofs: Whole Inventories, Charges, and Source-Native Continuation

高健（Jian Gao）

独立研究者 · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

摘要

本补充逐项展开正文第 4、5、7、8 节使用的完整接口：原回执的四支数学处置、全部 Type 库存的拆并提升、完整运输余项、原语法及有偿关系、整数系数与绑定体、前缀与实际调用、全计数准入、有类型库存、真实内容更新及同一运行的全部共尾片段。原根残差到付款与完成回答、长计算到下一纠正的完整证明分别见正文第 9、14 节。共享定理引用已发表过程核心第二版 DOI 10.5281/zenodo.23297175，本补充展开它们在完整账上的实际消费。

A 同一原回执的四支处置

A.1 完整恢复与三种准确残差

固定原动力学源 D、实际状态 s 和加法交换群目标 A。本源的输入律在每个原事件上给生成元类型 G、以该发生为根的完整曝光、种子、继续树及评价器树；种子与继续树生成完整历史 H_s。这些输入不含处置分支、kernel 零、coverage、逆映射、目标或 next。

关键的来源动作先发生：消去同一次原整账编译结果。它的 nativeWrite 载荷就是原 $\epsilon = D.\mathrm{emit}(s)$；其余四种结构载荷在此具名词汇为 PEmpty。以这枚 ϵ 读取源种子、continuation 和 evaluator；原事件呈现的往返等式给 receiptFromCompiler=ϵ。因此下面的代数对象都来自旧账中同一转移回执。

令 $E : \mathbb{Z}^{(G)} \to A$ 是原 evaluator 的线性延拓。源历史保实际生成元闭包与关系闭包。关系可靠时，E 消去这些关系并生成完成载体上的映射 $\overline{E} : \widehat{H}_s \to A$；若不可靠，必有实际关系 q 属源关系闭包而 $E(q) \neq 0$。原 faithful settlement 先作这个可靠性分类；可靠分支再内部检查核与余核。其四支为：

1. **忠实支**。关系可靠、完成映射核为零且实际余核为单点，源生成完整线性等价 $\phi : \widehat{H}_s \simeq A$。任意 $a \in A$ 的恢复是 $\phi^{-1}(a)$，其原评价 $\overline{E}(\phi^{-1}(a)) = a$。这条全称恢复是正向输出。
2. **关系不可靠支**。生成原关系 q 及准确坐标 $u = E(q) \neq 0$。关系属于本历史，非零像与原评价器是同一等式。
3. **核残差支**。关系可靠而 $\ker \overline{E} \neq 0$，生成 $k \in \widehat{H}_s$，$k \neq 0$ 且 $\overline{E}(k) = 0$。它是源完成的非零坐标，数值零没有将它删除。
4. **覆盖残差支**。关系可靠、核为零而 $A/\operatorname{im}\overline{E}$ 非单点，生成非零余核类 c 及原代表 $a \in A$，满足 $[a] = c$ 且 $a \notin \operatorname{im}\overline{E}$。对任意函数 $f : A \to \widehat{H}_s$，都有 $\overline{E}(f(a)) \neq a$。

这枚总处置是 noncomputable 数学构造：不可靠支从原不可靠证明提取关系见证；核非零支从原核子模取非零成员；核为零且未忠实时，若余核单点便会与原未覆盖证明矛盾，所以余核非单点，取两个不同类之差得到非零类，再用商映射满射取得代表。若该代表在原 range，它的商类为零，与所选非零类矛盾。内部先选核分支；不要求所有不忠实面同时交出核、余核两个见证。

直接消费者的完整证明。固定原 s，消去源刚生成的四支处置。忠实支直接用 $\phi\phi^{-1}=\mathrm{id}$ 得对全部 a 的恢复等式。不可靠支直接读取 $u\neq 0$。核支读取 $k\neq 0$ 和 $k\in\ker\overline{E}$，核成员给 $\overline{E}(k)=0$。覆盖支任取 f，若 $\overline{E}(f(a))=a$，则 $f(a)$ 连同该等式构成 $a\in\operatorname{im}\overline{E}$，与原代表的 range 排除矛盾。全过程没有将线性 recover 假设用于最后一支，故任意非线性函数也被该准确代表反控。

原 material 与 residual 投影在发射器前加入同一源；安装保持原 ledgerRoot，原发射器仍为 D.emit。所以与任一分支结论同时交出的旧行处置仍是

$$\mathrm{entry}(s)\xrightarrow{\text{原 receipt }\epsilon}\mathrm{entry}(D.\mathrm{update}(\epsilon)),\qquad \mathrm{next}(s)=D.\mathrm{update}(\epsilon).$$

其同债预算读取为原转移的非增等式，没有增加数学残差分类的 strict 付款。其四支类型与消元保持上述原输入。

B 全部旧类型、一般拆并与完整余项

B.1 旧类型的逐字段提升

先固定完整-原生原程序及其完整行列表 $I=\mathrm{Fin}(m)$。扩充索引取 $\mathrm{Fin}(m+1)$：每个旧位置用 castSucc 保留，最后一位是新数学行。旧原／目标项、两个选择器与完整回执逐位置复用；重复内容不会删位置。原两端 injectivity 只由旧 IdentityScope 证书和旧／数学标签互斥推出，不从覆盖推出。

一般拆并不再要求目标 IdentityScope。令 V 是母源的责任词汇，扩词汇的 content、residual、bearer、beneficiary、scope、event、admitted bearer ／ scope、谱系都带旧值／ math 标签。每个旧值 Type 纤维——原准入类型、authority、assumption、origin、discharge jurisdiction，以及所有进展／转移／终结字段——保留原类型。嵌入 $j:\mathrm{Obligation}_V\to\mathrm{Obligation}_{V^+}$ 逐字段加入旧值标签；部分读回 p 逐标签消去，旧值分支重组原完整记录。因此

$$p(j(a))=\mathrm{some}(a),$$

连同原 Type 字段全部成立；j 单射由对相等作用 p 后消去 some 得到。

母拆分回执的 children 列表变为 $\mathrm{map}(j)$ 原 children；合并 parents 同理。列表成员使用刚才的读回得到原成员，故每项源 ownership、anchor transport、谱系与 local discharge 正是原字段沿标签等式的运输。完整覆盖从原源纤维读取：旧目标落入相同 origin 时用旧值 embedding 单射反解原 origin 等式，调用原 covers_ 目标，再 map 入新列表。math 目标不可能与旧来源同标签。no phantom 从列表成员恢复母 child，再调用母 no phantom 给准确原目标并嵌入。child 严格预算下降沿原预算读取交换式运输。merge 的证明完全对称，逐原父项保其履责字段。

最后对扩账中的每对目标／源逐标签分析。旧值/旧值调用母证书的 identity ／ split 或 identity ／ merge；旧值/math 与 math/旧值被标签排除；math/math 是同一独立行。由此自产完整重组证书，其原证书来自同一实际 compiler image，经原 fold 与目标 occurrence 等式运输。消费者读取的就是这个证书，目标没有提交另一个分类或新覆盖。

完整 continuing 情形以原法给的逐当前 Packet 程序为输入。Packet 从同次 generatedLedger 消去原生、relation、continued、redirect 之一，保存原载荷、完整演化、原 finite patch 和实际 next。原数学动作与这一 Packet join：数学已付分支用其实际严格步，数学结算分支用原 Idle 演化；旧去向与 origin 分别交换：

$$d^+(j_s e)=j_t(de),\qquad o^+(j_t f)=j_s(of).$$

这些是带演化见证的完整依赖对，不只项名相等。原承担行的后继取 $d(\mathrm{owner})$ 后沿目标-emitted 等式运输。

生成片段仍只显式列一数学行，全部旧项交同源 remainder。数学去向 selector 命中零号，旧去向和所有 origin selector 选择余项；remainder 事件编译为这次完整账 join，并逐对发射精确 row 证据。逐标签计算两侧完整 Sigma，得 fold= 完整账 join。母完整重组证书随此 fold 提升；实际拍直接消去相同编译结果，所以其完整去向、原 projection outcome 及实际 next 同时相等。一个 Packet 不能替代逐当前程序；这个输入已明列。

C 有偿语法与自己的下一请求

C.1 执行与完整账

固定任意多型加法交换群值语言、变量环境 ρ 与源表达式 e。剩余计数逐节点定义为

$$r(c)=0,\quad r(x)=1,\quad r(e+f)=r(e)+r(f)+1,\quad r(Le)=r(e)+1,\quad r(B(e,f))=r(e)+r(f)+1.$$

变量绑定、两个常量的加法、线性与双线性运算各付一原语步；有上下文的步骤只把已生成的那一步送入左／右子式。逐构造子检查得每个实际步骤 $s:e\to e'$ 满足 $r(e)=r(e')+1$，同时携带 $\mathrm{ev}_\rho\, e=\mathrm{ev}_\rho\, e'$。左到右执行器先执行子式、再执行父节点；对表达式归纳，得到

$$T_e:e\to \mathrm{const}(\mathrm{ev}_\rho\, e),\qquad |T_e|=r(e).$$

这枚执行迹本身生成关系词：单步关系像为 $\delta_e-\delta_{e'}$，串接取和。对执行迹长度归纳，中间项相消，因此对每个已付前缀 $T:e\to e_k$，

$$\partial\eta_T=\delta_e-\delta_{e_k},\qquad \mathrm{ev}_\rho(\partial\eta_T)=0.$$

相消发生在关系像；实际 T 的步骤、顺序与关系证据仍在。

无旧承担者的构造以源根、原发生与原输入 reader 为输入。把原支撑扩成（原支撑，数学状态），活账分成全部旧项与一项独立数学责任。数学状态保存 (e_k,T_k)，不能只保存终值。原选动作在常量处生成局部结算，其余处发出执行器第一步。完整写入在旧分支读取同源余项，数学分支送到由本步产生的 $(e_{k+1},T_k\cdot s_k)$。有限片段只有一条数学行；余项实际编译为完整旧账演化。对两侧依赖对逐分支消去，片段的去向／ origin 与完整写入相等。

设 $B=r(e)$，原进程第 k 次访问的数学预算为 $B-k$。基点是空执行迹；付款步使执行迹加一且预算减一，局部结算处完整状态保持。因此

$$b_k=B-k,\qquad |T_k|+b_k=B.$$

实际数学行的准确去向就是下一访问数学行；旧／数学主张两种标签互斥，使同债目标唯一。逐行演化生成谱系，原动作的严格下降生成付款。每个 $k<B$ 有实际同债付款；$k=B$ 时 T_B 长 B，端点值等原 $\mathrm{ev}_\rho\, e$，再无该数学预算的付款宏步。由原执行得到的局部结算继续按其源合同使用。

十节点 Fock 实例另读实际当前物理环境 ρ_d：原执行迹属原环境 ρ_0，其余预算为 $10-(d+1)$，执行迹长度加预算为十。原 physical pair 从当前实际 occurrence 读出，不由已付数值重建。若当前 pending 为 e_d，更新残差恰为

$$\mathrm{physicalPair}_d-\mathrm{ev}_{\rho_d}(e_d).$$

预算为零时 e_d 已是原完整 pair 的常量，故该值变成当前态 pair 减 origin pair。此相等保留两份 pair 的完整分量与两个实际环境。

共同执行与无旧承担者定理具名引用核心命题 3.7、定理 6.10（S0.1–S0.3、S1）；本篇给出上述完整账与物理当前消费者。

C.2 整数、替换、余链和调用

整数系数。定义 $N_0(e)=0$，$N_{a+1}(e)=e+N_a(e)$。对 a 归纳得求值 $a\,\mathrm{ev}(e)$，预算 $a(r(e)+1)$。负整数 $z=-(a+1)$ 多执行一原负号线性节点，所以

$$c_z(e)=|z|(r(e)+1)+\mathbf{1}_{z<0}.$$

源词的有限支撑列表 L 依原顺序生成：空列表为常量零，$(e,z)::L$ 为 $E_z(e)+F(L)$。再对列表归纳得

$$r(F(L))=\sum_{(e,z)\in L}(c_z(e)+1),\qquad \mathrm{ev}(F(L))=\sum_{(e,z)\in L}z\,\mathrm{ev}(e).$$

执行器给等于此预算的执行迹。系数求和后同值／零值不删除实际表达式与费用。

binding body。设 θ 把原变量送到目标源表达式，固定目标环境 Θ，并令原环境 $\rho(x)=\mathrm{ev}_\Theta(\theta(x))$。对这枚 ρ 下的原步骤与原迹作代换，目标迹在 Θ 下执行。原单步的 subst 执行迹逐构造子生成：bind x 分支执行整个 $\theta(x)$，常量运算分支仍执行对应一步；上下文分支只把所生成执行迹送入原上下文。原串接执行迹通过 append 组合。故代换后执行迹真实连接 $e[\theta]$ 与 $e'[\theta]$，其关系边界仍伸缩为两端点差。常量终点时

$$|T[\theta]|=r(e[\theta]).$$

母引擎从相同 raw pair 生成的 normal 与完整账／ stage receipts 沿完整 HEq 读回；执行 binding body 不是以一个回调代收全部费用。

有序作用词。令 $w=\ell_1\cdots\ell_n$。原 sourceProgramme 每 letter 加入一线性节点，原变量一节点，模型投影再一节点，费用 $n+2$。保完整 prefix 再作后续作用的查询增加一节点，费用 $n+3$。E ／完整未来模型整对的总程序预算 $2n+5$。源效应 history 递归保存每 letter 作用前的 point、原单步残差及对应实际执行迹；空词执行费五，每个单 letter 执行费七，因此 historyCost 递归为

$$h([])=5,\qquad h(\ell w)=7+h(w),\qquad h(w)=7n+5.$$

完整库存同时嵌入整词执行迹与逐 letter history；$2n+5$ 与 $7n+5$ 是两个原程序的费用恒等。残差递归在后一段传播下满足 $R(\ell w,x)=U_w r(\ell,x)+R(w,A_\ell x)$；归纳给出完整 ordered residual 之和。末值相消不消除上述两个执行迹或原位置。

cochain 全坐标。给定表达式 e，List 取 $[\mathrm{updated}(e),\mathrm{old}(e)]++\mathrm{mixedTerms}(e)$，索引为其长度的 Fin。mixed 位置映射 $i\mapsto i+2$ 单射，重复表达式仍占不同位置。向量 query 对每项加一个坐标嵌入和一个累加节点，读回三个族分别为 updated 值、旧值、每个 mixed 值；原 mixed 更新定理于是成为同一向量的精确方程。其费用是所有项原费用之和加两倍 List 长度，执行迹和 relationWords 由原执行自产并写入 typed Born。

prefix 与 batch。取原 AST 源种子 e，$e_i=e[\theta]^i$。第 i 个原 prefix 的配对提升保预算，故实际 T_i 长度为 $r(\tilde{e}_i)$。令

$$\Sigma_n=\sum_{i=0}^{n}r(\tilde{e}_i).$$

源 batch 使用 $\mathrm{Fin}(n+1)$ 按每个位置加入坐标嵌入与累加节点；对其列表归纳，恰有

$$|T_{\mathrm{batch}}|=\Sigma_n+2(n+1).$$

每坐标读回原 prefix 旧值／效应整对，所有 prefix exposure 均经源种子入完整 inventory。

实际 calls。令 B 是这枚原源种子的 finite variable bound。源 calls 索引为 $\mathrm{Fin}(B+2)$：第 j 项在原 Future runtime 的 $c+j$ 拍读取自己的完整 callFrame、原输入、query、Result 与执行迹。observations 是这些原输入的原 environment；snapshotOld 读 ρ_{c+j}，snapshotIncrement 读 $\rho_{c+j+1}-\rho_{c+j}$，$j>B$ 时按原 finite 变量合同取零。对变量位置消去有界分支，实际 _node 环境读回逐项认回原 finiteOld ／ finiteIncrement，故 batch query 值相同。与此同时，每个 call 自有费用

$$\mathrm{callCharge}=\sum_{j=0}^{B+1}|T_j|=\sum_{j=0}^{B+1}r(\mathrm{Raw}_j)=\mathrm{callCost}.$$

此式不等同于 $\Sigma_n+2(n+1)$。后者是 snapshot batch 自身 AST 的费；前者是实际发出的每个调用的费。各 call 的每个 primitive 由其 callReader 的 OwnerFree.activePayment 产生严格扣减和原谱系；新的 batch 执行迹另有同债付款与 written Born/cofinal 嵌入。

实际 child。仅执行原旧值／ next 所选 child。未生成 successor 的 child 为零常量，费零；有实际 child 时原子树原输入费 $4m$，输出注入再加一，因此 childCost 为 $4m+1$。主 AST 取原 query、旧值 child、next child 之和，query 输出嵌入一节点、两次加法两节点，所以

$$c_{\mathrm{main}}=c_{\mathrm{query}}+c_{\mathrm{oldChild}}+c_{\mathrm{nextChild}}+3.$$

旧值 actor 库存、原 child relation 执行迹、lifted childTrace 及本次主 paidTrace 各自通过保顺序的 map ／源种子进入 stock；源-dependent next 沿原回调继续。

默认查询也按这条纪律消费实际子树：完整 Branch 核与关系两 AST 的联合费为两费之和加 4；complete actor 扩面增加一个节点；每枚已选 child 有自己的原成本 $4m_i$，输出注入后为 $c_i=4m_i+1$；没有源后继的分支是常量零，成本为 0。主查询的费用为原查询费加两项 child 费加 3。逐节点展开这几个语法构造给出等式，实际执行迹给出对应长度；旧 actor 库存、child 关系树、主已付暴露按原序进入同一库存。完整 Past、新 actor 和新的数学付款一并进入下一询问，费用没有在默认调用处变成免费的注释。

C.3　原数学行的全部计数准入

数学 compiler 的初始片段自身选中数学行。canonicalGeneratedEntryRow 的成功由这枚实际 patch 的选择等式推出，遂从 initialRow 生成 sealed 初始 authority。对 k 递推：已得 (e_k,a_k)，则下一项是同一完整账 write 的 canonicalTargetEntry，下一 authority 是 a_k.next；不从目标侧重新准入。这给全部 count 的原行及权限。

完成材料保存 $M=(\rho,e,e',T,\mathrm{owner},\varepsilon)$。下一残差请求为 $e+(-e')$，在 $\rho+\varepsilon$ 执行；可靠性给旧值零，更新等式给新值 $\mathrm{eff}(e)-\mathrm{eff}(e')$。其 AST 预算

$$r(e)+r(e')+2>0$$

保证 first 动作是原 primitive 付款步。由这一步、原登记输入及源 authority 产生责任出生程序，其 compileInquiry 在同一 exact event 上等于 debtAdmission。目标的 firstDestination、全部旧 projection outcome 和 literal next 都是该同一 generatedAction 的字段。给任意原 Frame、源 Programme 及独立 LowVar，Shared 运行继续所有 offset 实际 query ／ answer ／ next，同时沿数学行读不补充预算与无总律付款良基。AC 固定环境的增量为零时，请求仍保原 raw body 并付 $B+2$ 的语法责任。

D　有类型库存、内容更新与完整共尾

D.1　完整库存的依赖读回

完整材料含 physicalRaw ／ lowRaw、两份已付有偿状态／执行迹、逐变量 binding、migrated 执行迹及 written。原 stockLaw 在每个 supplied occurrence 直接输出这个完整依赖记录。母目标的 inherited outcome 与源 outcome 有 HEq；同一 HEq 给载荷类型相等，沿这条类型等式 cast 原目标 outcome，然后消去和类型：inactive 是 PEmpty，active 投影返回原 completeMaterial。cast 的异构等式与原 HEq 复合成为普通相等，因此取出的 stock 确等于原记录。这里没有从字段名或同值推断类型。

physical 事件按原 binding 逐式／逐关系词迁入 low 语言；旧值与 low 通过源种子左右注入保重数。原源 written exposure 和实际 query exposure 先以完整树等式认回，再嵌入 Born。对本次 query 执行迹的位置，原三个源种子接缝给

$$\iota(j) = j + |H_{\mathrm{old}}| + |H_{\mathrm{physical}}| + 3, \qquad H_{\mathrm{Born}}[\iota(j)] = H_{\mathrm{query}}[j].$$

自然数取消同偏移给单射，加同偏移给严格保序；不能据同值删不同位置。这一准确偏移限于该具名 query-to-Born 树形。

完整 epoch inverse 写回另有源 fold。第 d 个 stage 先用 literal next 环境执行当前 syntax，生成 nextTrace ／ nextWord，再执行其配对逆读；written(d) 保旧 pair 及该新已付 exposure。historyWritten(0)=written(0)，historyWritten(d+1)= 源种子 (historyWritten(d),written(d+1))。按 last 归纳，任意 $0 \le d \le$ depth 的任意原 event 均在 historyWritten(depth)；对实际 next 分支消去，mathNext 保原库，nextBorn 取同 sourceWritten。完整库因此包括所有 epoch，而非只留最新执行迹或 normal。

D.2　出生的真实内容

配对提升的实际结果为 $(\mathrm{ev}_{\rho}\, e, \mathrm{eff}_{\rho,\varepsilon}\, e)$。两分量相加由源更新恒等精确等于 $\mathrm{ev}_{\rho+\varepsilon}\, e$。实际出生环境读口的原解码写入器读取这份已付 pair，nextBorn 的 environment 字段按定义就是它的读出。因此出生推进了实际内容，仍同时携旧 scalar、pair 和本次完整已付 exposure。

原 History 程序在 active 环境中的输出作为下一 environment；再用同一构造于这一 literal Born，得到第二次实际程序输出。原 history(stage) 的种子已经提供 stage+1 活环境，所以下一次出生读为 history(stage+2)。两次更新来自两次自己的原执行迹与发生，不把同一结果重名为第二次执行。

源-family 的 typed child 由原 Move 递归生成 nextSeed ／ nextCfg；Admission.generatedAction 同口给目标 root、完整账 first、旧 projection、源 valid 和下一 reader。Born.next_environment_ 源将环境完整认回同发生的 physicalNext。不同发生的环境沿各自等式相接；Some 实际解码写入器与 none compatibility 分支按原代码分别使用。

D.3　原 Type 回执的准确距离

固定任意有效源程序、原 Frame 及起点 i，令 B_i 是实际 pending AST 的剩余计数。回执返回 Type 中的 $(d, s, \text{actual action at } i+d = \mathrm{settled}(s))$，连同 $d \le B_i$。按原动作消去：数学结算返回距离零；已付使用准确 mathNext，预算减一，递归只在这个下一 Frame，距离加一。已付分支等式与索引 $(i+1)+d' = i+(d'+1)$ 同时运输终点。

对 $h \le B_i$ 归纳，$B_{i+h} = B_i - h$；若 $h < B_i$，预算正，数学结算分支不可能，故取得原实际已付 Step。回执终点预算零且 $d \le B_i$，代入得 $d = B_i$。执行迹逐步 append 给全部已付前缀，可靠性给准确端值。原 compiler 在回执指明的同次数学结算事件实际发射 debtAdmission，其 next 节点完整认回 nextBorn。完整 stock 的位置嵌入由 D.1 付清。

D.4　同一运行与全部未来

原 StockTarget 的 compile 函数直接匹配实际动作：已付送 answered 并取 mathNext，数学结算送原 birthProgram 的 debtAdmission 并取 nextBorn。两支分别展开编译匹配，产生分支等式；successor_valid 分别调用 directlyAnswered 或 debtAdmission 的原法则，并用完整 targetRoot ／ targetCurrent 等式认回下一 presentation。不能把数学结算沿 answered 分支擦掉。

Stage 保存语言级、完整包、configuration 和 language；continuation 还保存原已付 Frame。唯一 runtime 在拍 t 的实际 _node 归纳等于这个 Stage 在 t 的 presentation。每拍 query、branch-specific 回执与完整账均从原拍读取；语言级不替代拍。

令 $d(t)$ 为不早于拍 t 的首个准入距离。运行初点为准入，故 $d(0) = t_0 = 0$。从任意 start 消去 Stage: admission 给 distance 零；continuation 用 D.3 给 B 个已付前缀、数学结算终点，再经过原下一拍进入 admission，distance 为 $B + 1$。所有中间 Frame 等于同 runtime 相应索引的 Frame。令

$$t_0 = d(0), \qquad t_{j+1} = t_j + 1 + d(t_j + 1), \quad \ell_j = 1 + d(t_j + 1).$$

所以 $t_j + \ell_j = t_{j+1}$，$t_{j+1} > t_j$，归纳 $t_j \ge j$ 给任意 bound 的共尾覆盖。historyAt(j) 是原拍 $t_j + h$ 的完整 runFrom，含准入拍及全部 between-stop 已付／数学结算 ticks。packetAt(j+1) 完整等于原 receiver→ 回执 →birth 的包，只有在这次实际出生语言级才加一。

每 stop 的材料 M_j 保原输入、环境、有偿状态、全执行迹及原承担行，installed 原输入四字段沿原 full-当前态身份运输。执行迹关系词像 $b_j = \delta_{e_j} - \delta_{e'_j}$ 属同已付 history 的 relationClosure，六面由同一材料生成；literal-word 恢复对任意完整词成立。旧 σ 与新 environment 下重新执行生成的 σ 各自保在库存。

在 stop j 的完整原生词模 $\mathcal{W}_j$ 上，literal sourceWordAction 先配对提升，再沿真实下一包 language ／ full 完整包等式运输为 $S_j : \mathcal{W}_j \to \mathcal{W}_{j+1}$。原库存成员经 bornStock 嵌入，故实际 acted word 进入下一原库存。receiverRead 先逆运输这些等价再读原 receiver 环境，得到

$$\mathrm{receiverRead}_j(S_j w) = (\mathrm{ev}_{\rho_j^{\mathrm{receiver}}}\, w, \mathrm{eff}_{\rho_j^{\mathrm{receiver}}, \varepsilon_j^{\mathrm{receiver}}}\, w).$$

这是原接收器环境方块，各语言级环境不需预先相等。完整有偿状态的源片段仍按 D.4 前段在原拍上推进。

共同载体 $\mathcal{W} = \bigoplus_j \mathcal{W}_j$ 的 finite support 沿 $j \mapsto j + 1$ 保 finite，线性延拓得 S。局部旧值／效应读口 d_j 的定义域为 $\mathcal{W}_j$；站位投影 $\mathrm{pr}_j : \mathcal{W} \to \mathcal{W}_j$ 将它扩展为直和读口 $D_j = d_j \circ \mathrm{pr}_j$。全部允许字母是原单作用选择 PUnit；对其任意有限词 a 定义

$$K_{\mathrm{fut}} = \bigcap_a \bigcap_j \ker(D_j S_a).$$

加一个原字母仍在量词范围内，故此核对 S 封闭，商生 modelAction；类相等等价于全部词／站位观察相等。任何当前不可见且对 S 不变的子模按词长归纳都含于此核，故它最大。空词包含当前读口，使未来核含于原生 completion 核，从而产生合法 nativeRestriction；modelAction 经它读到原 S 的下一值。原完整有偿状态、CommonWords、receiver 环境与完整未来模型分别保其类型，由实际 consumer 同口使用。

D.5 差词自己的费用与逐回执总值

原 next 发生上的两份完整执行迹边界 b_{Ψ}, b_{μ} 各在原实际 history 关系闭包。其同 scope 读值为 $r - c(\Psi, 0)$ 与 $r - c(\mu)$，其中原 writer 的 $\mu = (\Psi, \sigma)$。相减后实际 scope 读

$$(r - c(\Psi, 0)) - (r - c(\mu)) = c(0, \sigma).$$

源差词独立生成 C.2 的系数 AST 并按其自己的 cost 执行，形成自己的 Result 与全执行迹；完整源 coface 的 supplied 材料随母准入进入真实 $k + 2$ 包，原完整账和所有旧投影同时保留。

同 Producer.result 的效应与 updated inverse 读取这份新执行迹。任意新已付 event 先入 pairWritten，再经原生 _ 已付 _in_stock 及 stock_in_ 联合进入实际 jointStock；这三个具名嵌入复合给每个发生的成员。实际 consume 同时使用此 carrier、原 outer reader、inner query 与完整账-next；值相消不删发生。

总值写回另以原同次完成材料为起点。原已付端点为 const(p)，$p = \mathrm{ev}_{\rho}\, e$，新原生请求 $e - \mathrm{const}(p)$ 在实际 afterEnv $\rho + \varepsilon$ 完成，值为

$$n = \mathrm{ev}_{\rho+\varepsilon}\, e - p, \qquad p + n = \mathrm{ev}_{\rho+\varepsilon}\, e.$$

原 cfg 只修改源-owned nextEnvironmentReadAt，保 reader 与双库存回调。实际回执选择数学结算事件，在该分支原解码写入器调用 write(p+n)，故真实 Born 写的是这枚回执的 total。原生 =0 时仍写 p。下一包重读它自己的实际 actionResult 得到 p^{+} 和 fresh 解码写入器，再重复上式；它不继续使用 p。共用证明具名引用过程核心第二版定理 8.14 ／ 8.15 与 S2.3 ／ S5.2，完整 H0 源环境修复按真实公开实现身份引用。

E 原长计算的源程序接口

E.1 原算术线性计算

对正文第 14.1 节列明的零点观察输入，原非平凡坐标与 right-half 证明纤维给出原 q_0、算子 R 和 Xi 精确阶 m。源 AST $e_0 = x$、$e_{k+1} = R(e_k)$，环境只装原 q_0。归纳得求值 $R^k q_0$、语法费 $k + 1$。执行器因此生成实际 targetState、value $R^m q_0$ 与完整 $m + 1$ 执行迹。component 在原发射器前安装，closed facade 的同发生读口直接取出这份 Result；relation、inverse 及 cochain 各消费此同执行迹。

原 lower ledger、发射器及 physical next 由同投影安装交换式保留，$m + 1$ 是原 AST 责任尺度。实际物理一拍另读原 root 的回执。原 physical membership 由领域原 Xi 消费者接在实际值之后；其解析推导归 RS。这枚实际完成值还生成常量环境增量、下一原线性残差及完整分裂的两分量。这个原程序接口为正文第 14 节的完整窗口与下一纠正提供实际供料。

参考文献

[1] Jian Gao. *The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes.* Version 2, 2026. DOI: 10.5281/zenodo.23297175. 正文及完整证明补充。